

# Self-Reflective Report

BIT1123 Object Oriented Programming — Assignment 1

**Name:** Khaled Saeed Ahmed

**Student ID:** 202309010806

**Course:** BIT1123 Object Oriented Programming

**Institution:** City University Malaysia

## 1. Introduction

This report reflects on my learning journey through the BIT1123 Object Oriented Programming lab tutorials, from Week 1 to Week 10. Over the semester, the weekly tutorials took me from creating my first GitHub repository and writing a simple Hello World program to building a complete graphical quiz application in Java Swing. The purpose of this report is to summarise what I learned in each tutorial, the problems I faced along the way, how I solved them, and how my understanding of Java and object-oriented programming developed as a result. All of the source code discussed here is available in my GitHub repository, organised week by week.

## 2. Knowledge Gained from Each Tutorial

### Week 1 — Getting Started with GitHub and Codespaces

The first tutorial was about setting up the tools rather than heavy coding. I created a GitHub account and repository, added a README, a Java .gitignore and an MIT license, and learned the basic Git commands: git status, git add, git commit and git push. I then created a tutorial folder containing HelloWorld.java and StudentGrade.java, and learned how to compile and run Java programs from the terminal using the java command from the folder that contains the file. Even though the programs were simple, this week taught me the workflow I would use for every tutorial afterwards.

### Week 2 — Classes, Objects, Constructors and Methods

This week I wrote my first proper class. The Student class had attributes (name, age, gpa), a constructor to store values passed in from the Main class, and methods such as displayInfo(), study() and takeExam(). Completing the fill-in-the-blank template made me understand how a constructor's parameters are assigned to the attributes, and how an object is created with the new keyword and then used to call methods.

### Week 3-4 — Inheritance and Method Overriding

The scenario was a university that manages different types of people. I built a Person superclass with private fields (name, id), getters, and an introduce() method, then created

Student and Lecturer subclasses that extend Person, call `super(name, id)` in their constructors, and override `introduce()` to print their own message. Seeing 'I am a person.', 'I am a student.' and 'I am a lecturer.' printed from the same method call was my first real experience of polymorphism: the same reference type (Person) behaving differently depending on the actual object.

### **Week 5 — Encapsulation (Student Information System)**

This tutorial focused on data hiding. The Student class declared `studentID`, `name` and `cgpa` as private, and I completed the setters and getters, then added a fourth variable, `programme`. I also answered three questions in `documentation.txt` about why variables are private, what happens if they are public, and why we use getters and setters. Writing the answers in my own words forced me to actually understand encapsulation instead of just applying the syntax.

### **Week 6 — Employee and Lecturer (Inheritance with super)**

I created an Employee class with protected `id` and `name` fields and a `displayInfo()` method, and a Lecturer subclass that adds a `subject` field, passes `id` and `name` up with `super()`, and displays its subject. The activity also required adding a `department` variable. This week reinforced Week 3-4 but with the protected modifier, which showed me the middle ground between private and public when a subclass needs access to superclass fields.

### **Week 7 — Abstraction (Smart Home Appliance Management System)**

The scenario asked for a common interface for appliances while hiding the implementation details. I completed an abstract Appliance class with a constructor, `displayBrand()`, `turnOn()`, `turnOff()` and an abstract `operate()` method, then wrote two concrete subclasses from the suggested list: `WashingMachine` ('Washing clothes...') and `Refrigerator` ('Store food & beverages...'). The key lesson was that an abstract class cannot be instantiated directly, but it can force every subclass to provide its own `operate()` implementation.

### **Week 8-9 — To-Do List with ArrayList and File I/O**

This was the first program that felt like a real application. Using an `ArrayList` and a `Scanner`, the program lets the user enter three tasks, displays them as a numbered list, saves them into `task.txt` using `BufferedWriter`, and then reads them back with `BufferedReader` and displays the loaded tasks. I learned about collections that grow dynamically (unlike arrays), the `try-with-resources` syntax, and why `IOException` must be caught when working with files.

### **Week 10 — Programming Quiz Battle (Java Swing GUI)**

The final tutorial combined everything into a GUI game. The `Questions` class stores a question, two options and the answer, with an `isCorrect()` method that compares the user's choice. `QuizBattleGUI` extends `JFrame` and implements `ActionListener`: it creates labels and two buttons, positions them with `setBounds()` on a null layout, and responds to clicks in `actionPerformed()` by checking which button was pressed with `e.getSource()`. I also

practised the full Git cycle from the instructions: cloning my repository locally with git clone, editing in VS Code, and pushing the work back to GitHub.

### **3. Challenges Encountered**

The challenges I faced were mostly practical rather than dramatic. First, in the early weeks I did not fully understand the strict relationship between a public class name and its file name. Some tutorial sheets listed file names like `main.Java` or `sTudent.java`, and following them literally produces compiler errors such as 'class Student is public, should be declared in a file named Student.java'. Second, the tutorial templates are fill-in-the-blank, so a single wrongly completed blank (for example forgetting 'this.' in a constructor or using the wrong variable in a print statement) produced output that did not match the expected output in the sheet. Third, in Week 8-9 I initially had to work out which reader and writer classes to wrap together (FileWriter inside BufferedWriter, FileReader inside BufferedReader) and how the while loop condition (`line = reader.readLine()`) `!= null` actually works. Finally, in Week 10 the null layout meant every component had to be positioned manually with `setBounds()`, and getting sensible coordinates and a suitable frame size took some trial and error before the window matched the expected screenshot.

### **4. How the Challenges Were Overcome**

I overcame the file naming problem by adopting one simple rule: the file name must exactly match the public class name, so I standardised my files to `Main.java`, `Student.java`, `Employee.java` and so on, regardless of small typos in the sheets. For the template blanks, my method was to compile and run after completing each class and compare the console output character by character against the expected output in the tutorial, fixing differences immediately instead of writing everything first and debugging a large mess at the end. For file I/O, reading the provided skeleton carefully showed that the `try-with-resources` block closes the file automatically, and printing `e.getMessage()` in the catch block helped me understand what failed when the path was wrong. For the GUI, I adjusted the `setBounds()` values a few at a time and re-ran the program until the layout looked like the expected output window.

### **5. Improvements in Java Programming Skills**

Comparing Week 1 to Week 10, the improvement is clear. At the start I could only print text and use if-else statements to assign a grade. By the middle weeks I was comfortable writing classes with constructors, getters and setters, and small inheritance hierarchies. By the end I could use `ArrayList` and `Scanner` together, handle exceptions around file operations, and build an event-driven Swing interface. Just as importantly, my workflow improved: I now compile and test in small steps, read compiler errors instead of guessing, and keep every week's work committed and pushed to GitHub so nothing is lost.

## 6. Understanding of Object-Oriented Programming

The tutorials demonstrated all four OOP pillars in a practical order. **Encapsulation** appeared in Week 5, where private fields with getters and setters protect student data from being changed directly. **Inheritance** was practised twice, in Week 3-4 (Person, Student, Lecturer) and Week 6 (Employee, Lecturer), including constructor chaining with `super()` and the protected modifier. **Polymorphism** came from method overriding: a Person reference can hold a Student or Lecturer object, and the overridden `introduce()` method decides at runtime which message is printed. **Abstraction** was the theme of Week 7, where the abstract Appliance class defines a common interface (`turnOn`, `turnOff`, `operate`) while each appliance hides its own implementation details. Week 10 then showed these ideas working together in a real program: `QuizBattleGUI` inherits from `JFrame`, implements the `ActionListener` interface, and the `Questions` class encapsulates the quiz data.

## 7. Future Learning Plans

Based on this semester, my next steps are: first, to strengthen my Java fundamentals by practising more exercises with collections (`List`, `Map`, `Set`) and exception handling, since Week 8-9 showed me how central these are to real applications. Second, I want to learn proper layout managers (`FlowLayout`, `BorderLayout`, `GridBagLayout`) instead of null layout, and eventually try `JavaFX` for more modern interfaces. Third, I plan to extend the Week 10 quiz into a multi-question game with a score counter, as it is a natural continuation of the tutorial. Finally, I want to keep improving my Git skills, especially branching and pull requests, so that I can collaborate on group projects confidently.

## 8. Conclusion

The ten weeks of tutorials built up my skills in a logical sequence: environment and tooling, then classes and objects, then the four OOP pillars, and finally practical applications with collections, files and a GUI. The habit of pushing every tutorial to GitHub means I now have a complete, organised record of my progress, and this assignment itself was a useful exercise in presenting that work professionally. Overall, I finished the semester far more confident in reading, writing and debugging Java programs than when I started, and with a genuine understanding of why object-oriented design matters rather than just how to write the syntax.

## 9. GitHub Repository URL

<https://github.com/ksdsass/khaled-saeed-ahmed-202309010806-oop>